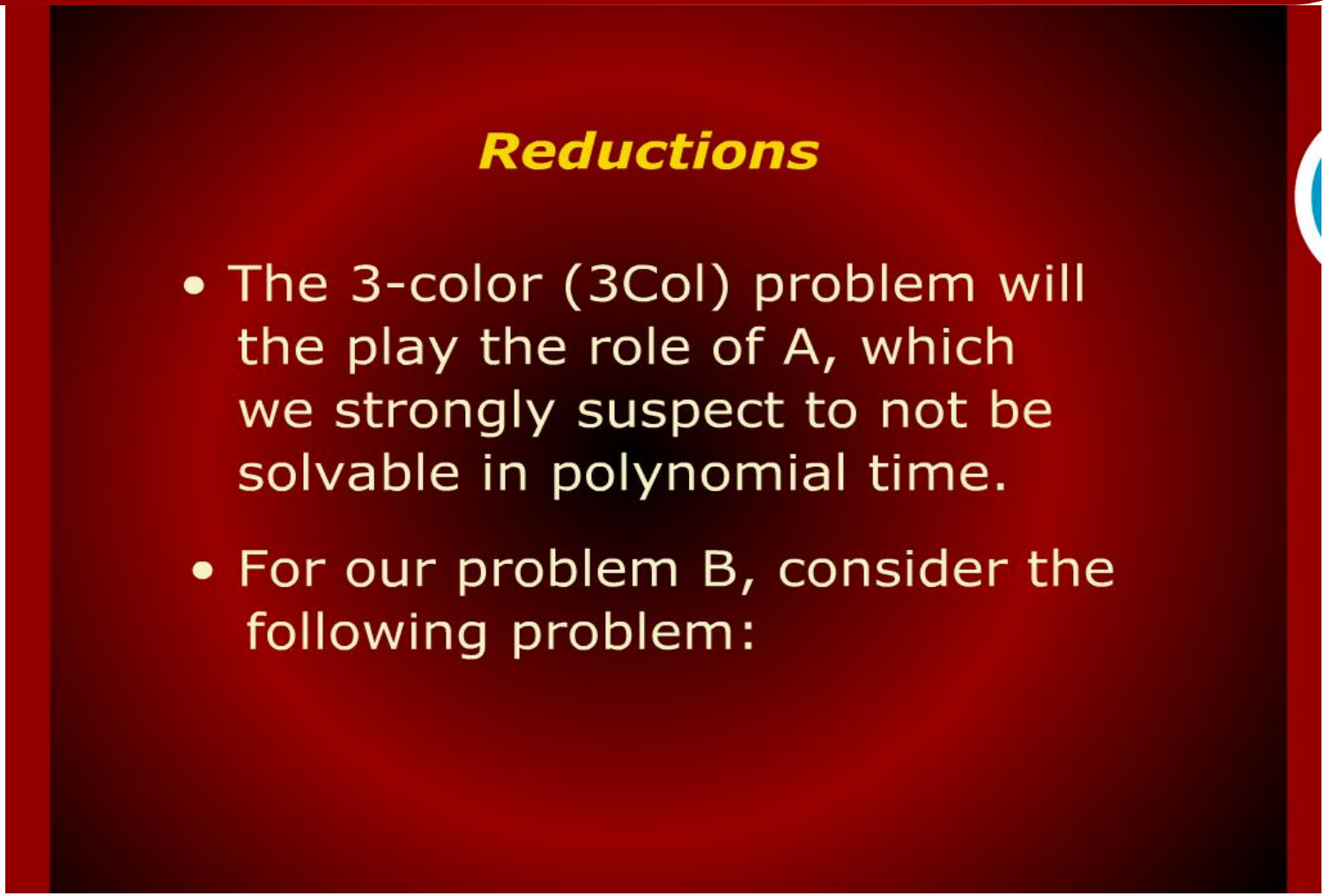
Algorithms

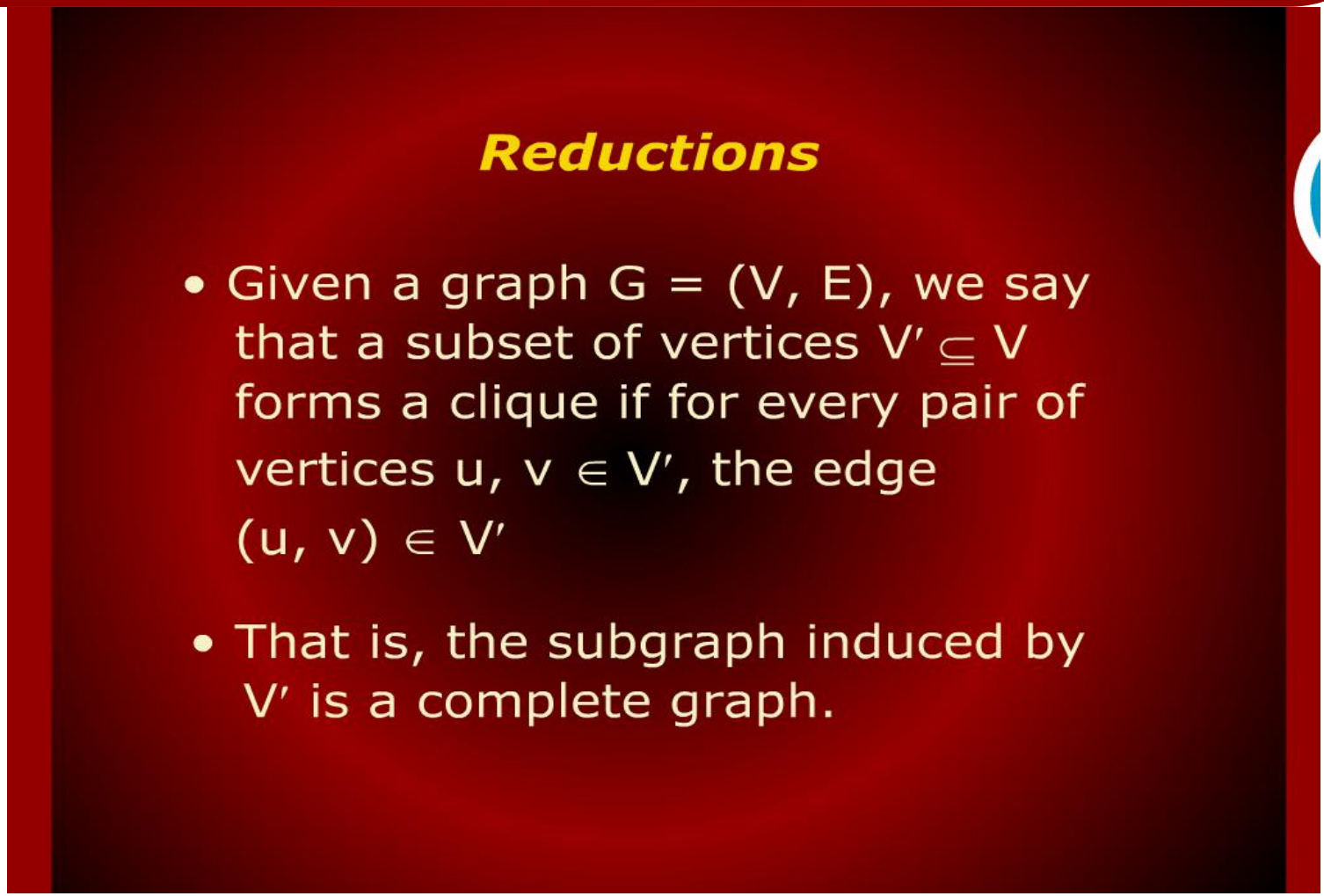
Lecture #45

Syed Hamed Raza

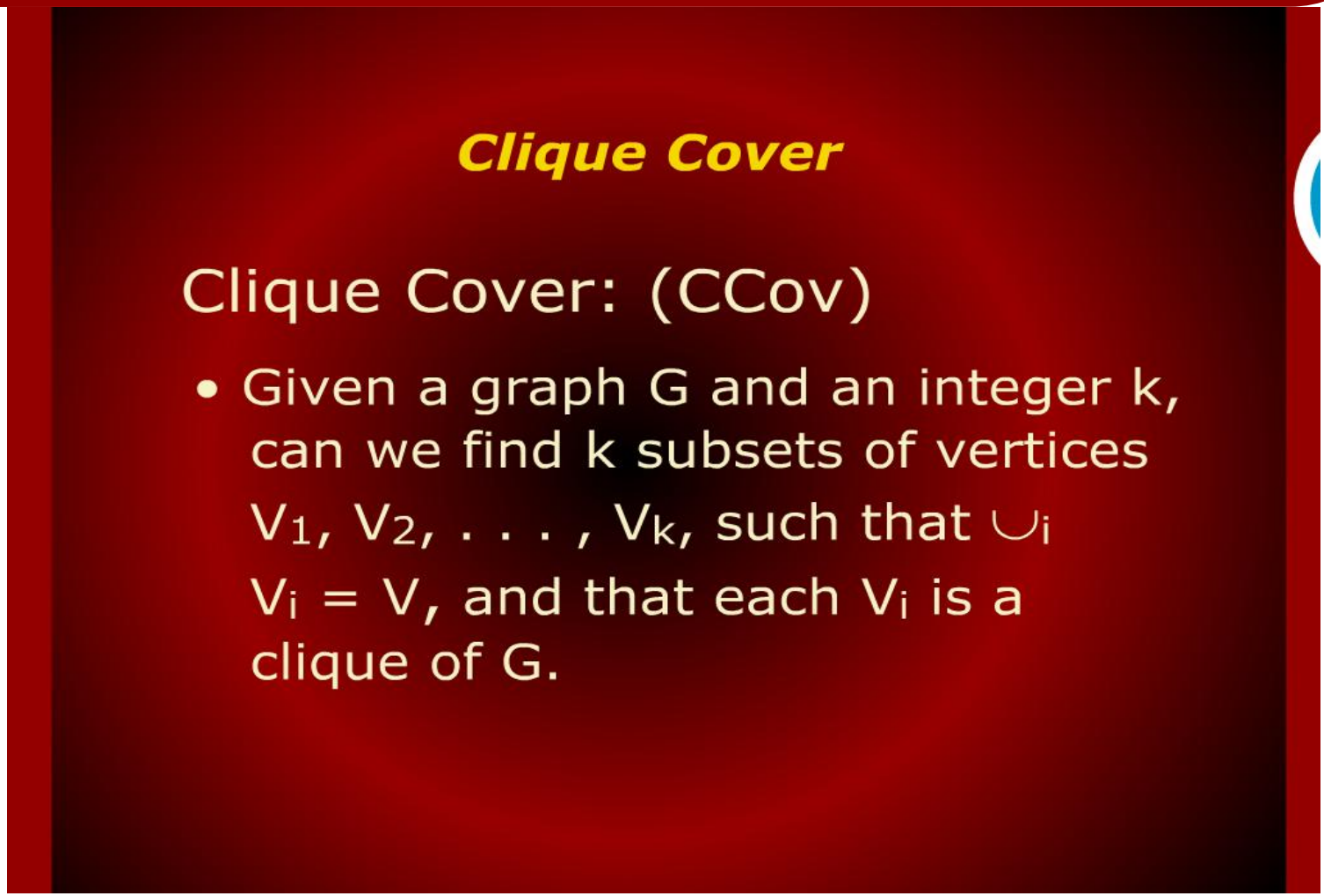
Reductions


Reductions
• The 3-color (3Col) problem will the play the role of A, which we strongly suspect to not be solvable in polynomial time.
• For our problem B, consider the following problem:

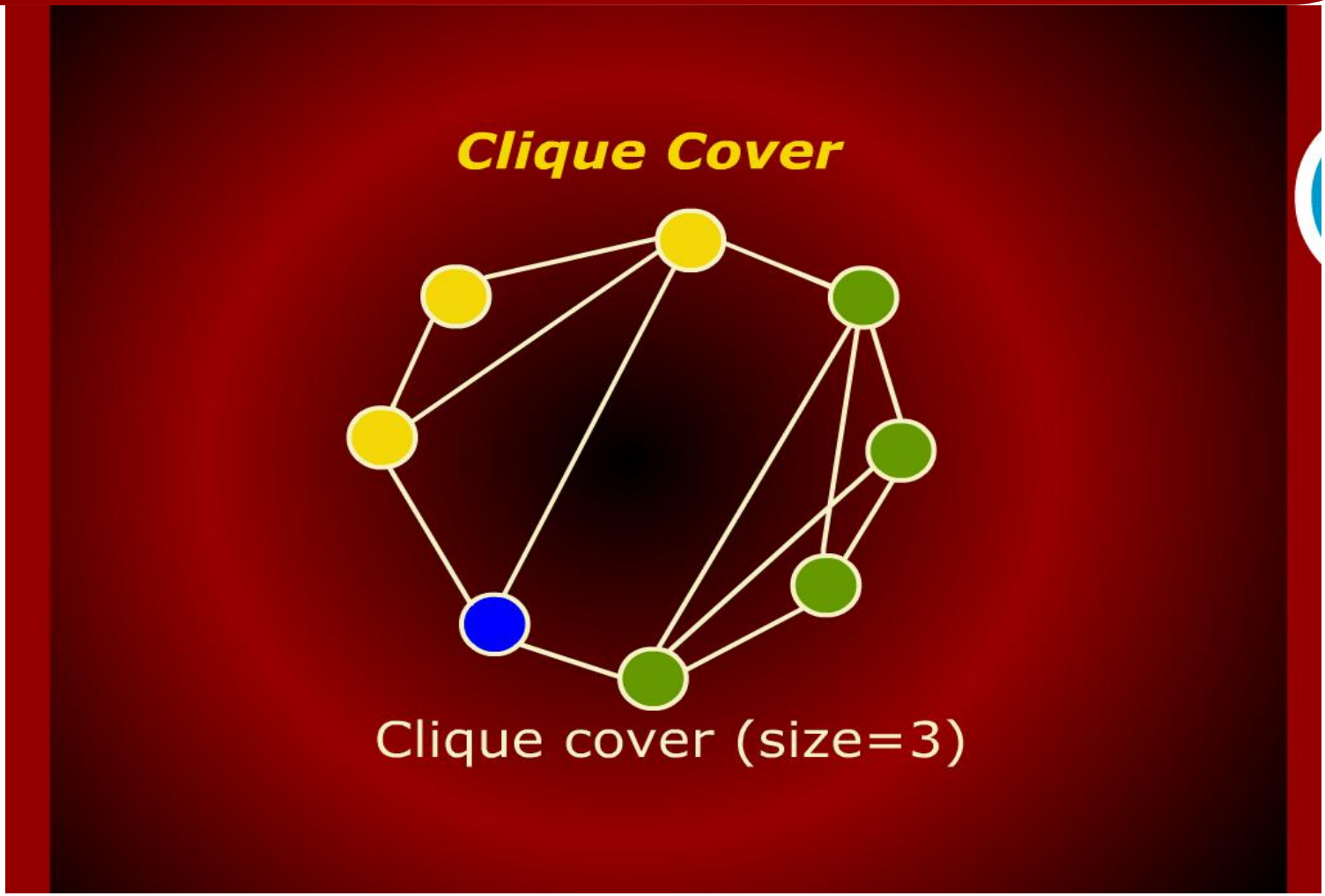
Reductions


Reductions
• Given a graph G = (V, E), we say that a subset of vertices V′ ⊆ V forms a clique if for every pair of vertices u, v ∈ V′, the edge (u, v) ∈ V′
• That is, the subgraph induced by V′ is a complete graph.

Clique Cover



Clique Cover

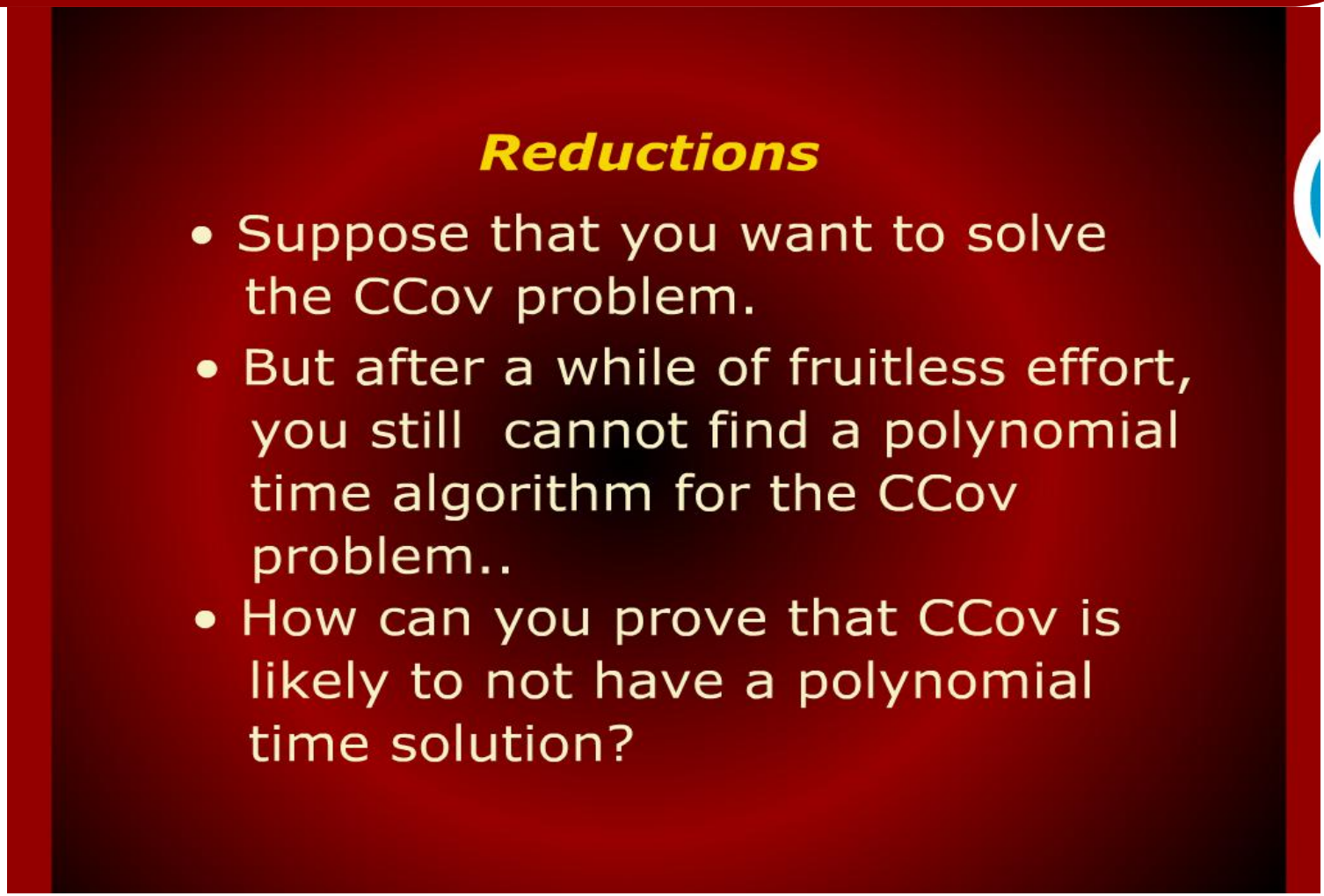


Clique Cover

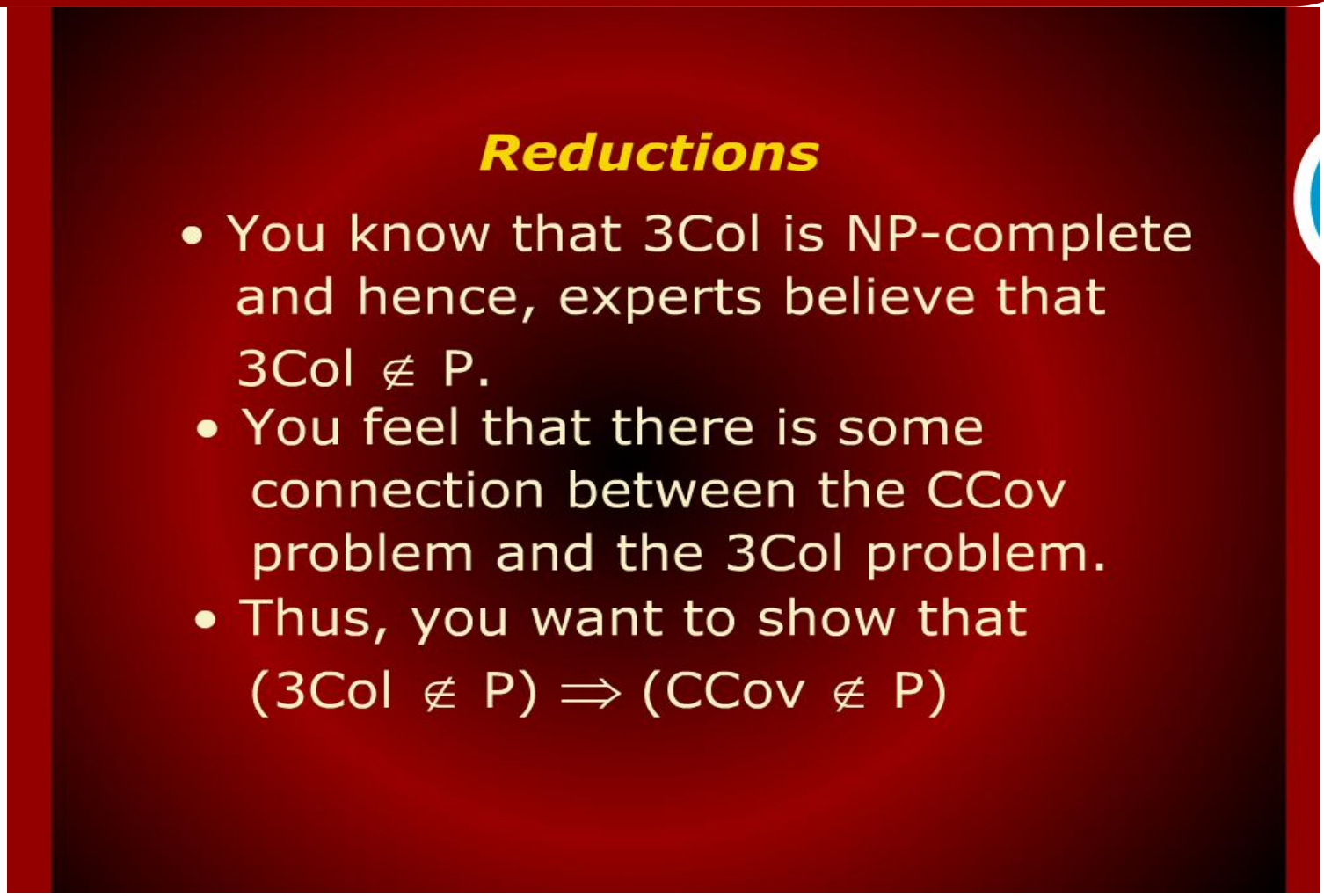
Clique Cover

- The clique cover problem arises in applications of clustering.
- We put an edge between two nodes if they are similar enough to be clustered in the same group.
- We want to know whether it is possible to cluster all the vertices into k groups.

Reductions


Reductions
• Suppose that you want to solve the CCov problem.
• But after a while of fruitless effort, you still cannot find a polynomial time algorithm for the CCov problem..
• How can you prove that CCov is likely to not have a polynomial time solution?

Reductions


Reductions
• You know that 3Col is NP-complete and hence, experts believe that 3Col ∉ P.
• You feel that there is some connection between the CCov problem and the 3Col problem.
• Thus, you want to show that (3Col ∉ P) ⇒ (CCov ∉ P)

Reductions

Reductions

- Both problems involve partitioning the vertices into groups.
- In the clique cover problem, for two vertices to be in the same group, they must be adjacent to each other.
- In the 3-coloring problem, for two vertices to be in the same color group, they must not be adjacent.

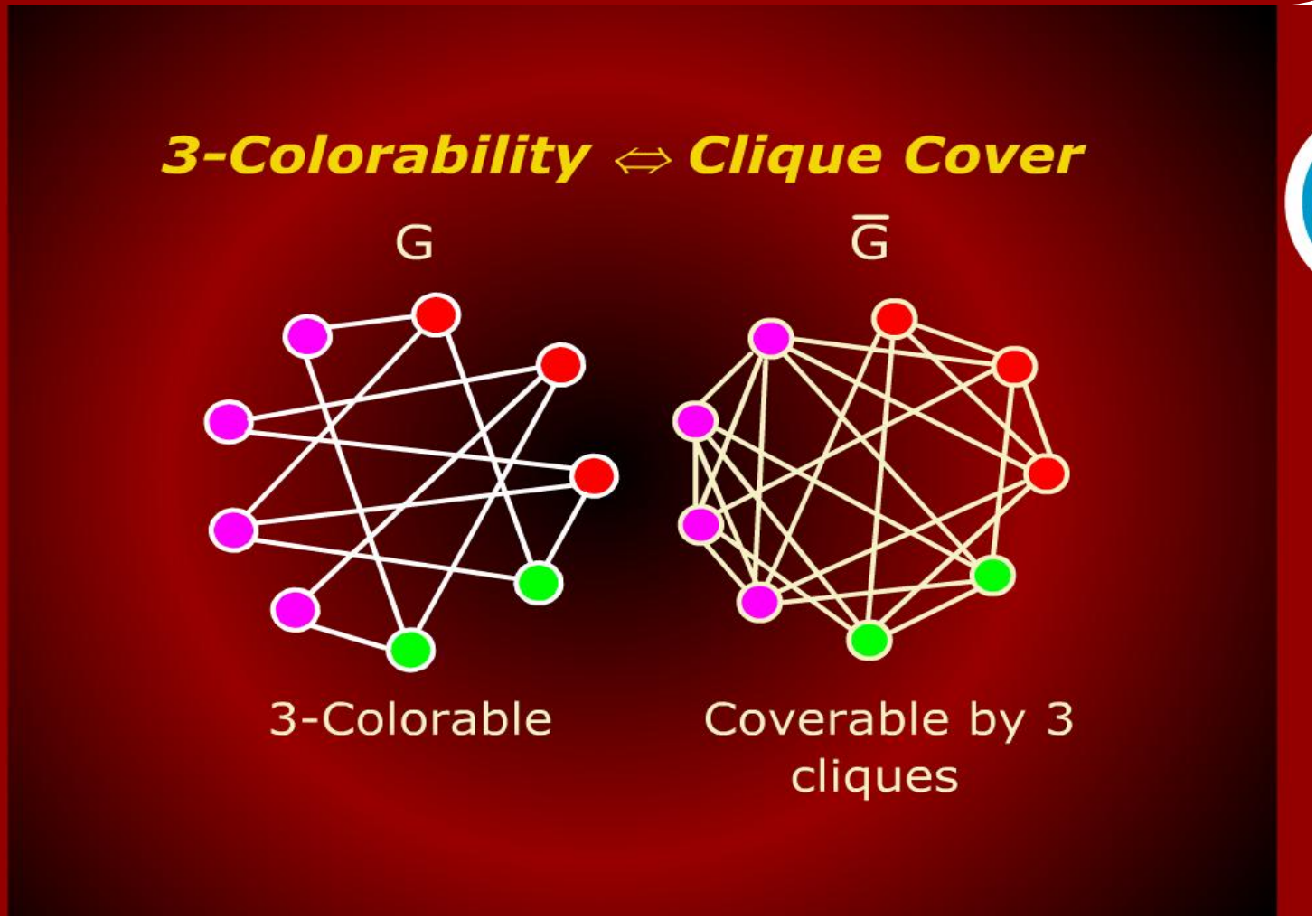
Reductions

Reductions

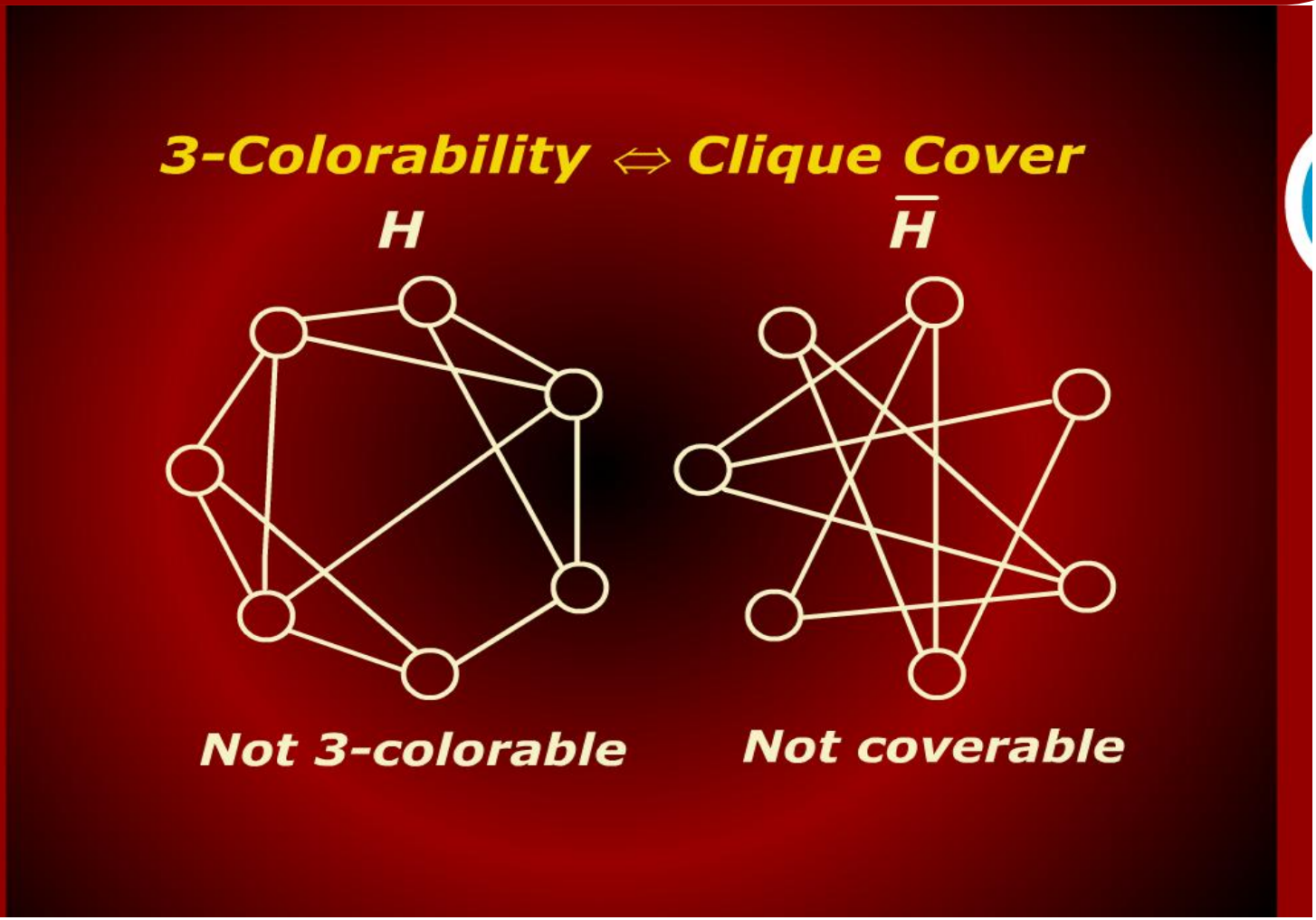
two vertices to be in the same group, they must be adjacent to each other.

- In the 3-coloring problem, for two vertices to be in the same color group, they must not be adjacent.
- In some sense, the problems are almost the same but the adjacency requirements are exactly reversed.

3-Colorability <=> Clique Cover



3-Colorability <=> Clique Cover



Polynomial Time Reduction

Polynomial Time Reduction

Definition:

- We say that a decision problem L_1 is polynomial-time reducible to decision problem L_2 (written $L_1 \leq_p L_2$) if there is polynomial time computable function f such that for all x, $x \in L_1$ if and only if $f(x) \in L_2$.

Polynomial Time Reduction

Polynomial Time Reduction

- In the previous example we showed that

  $3\text{Col} \leq_P \text{CCov}$

- In particular, we have

  $f(G) = (\overline{G}, 3)$.

- It is easy to complement a graph in $O(n^2)$ (i.e., polynomial time).
- For example, flip the 0's and 1's in the adjacency matrix.

Polynomial Time Reduction

Polynomial Time Reduction

- The way this is used in NP-completeness is that we have strong evidence that L_1 is not solvable in polynomial time.
- Hence, the reduction is effectively equivalent to saying that "since L_1 is not likely to be solvable in polynomial time, then L_2 is also not likely to be solvable in polynomial time.

NP-Completeness

NP-Completeness

Definition:

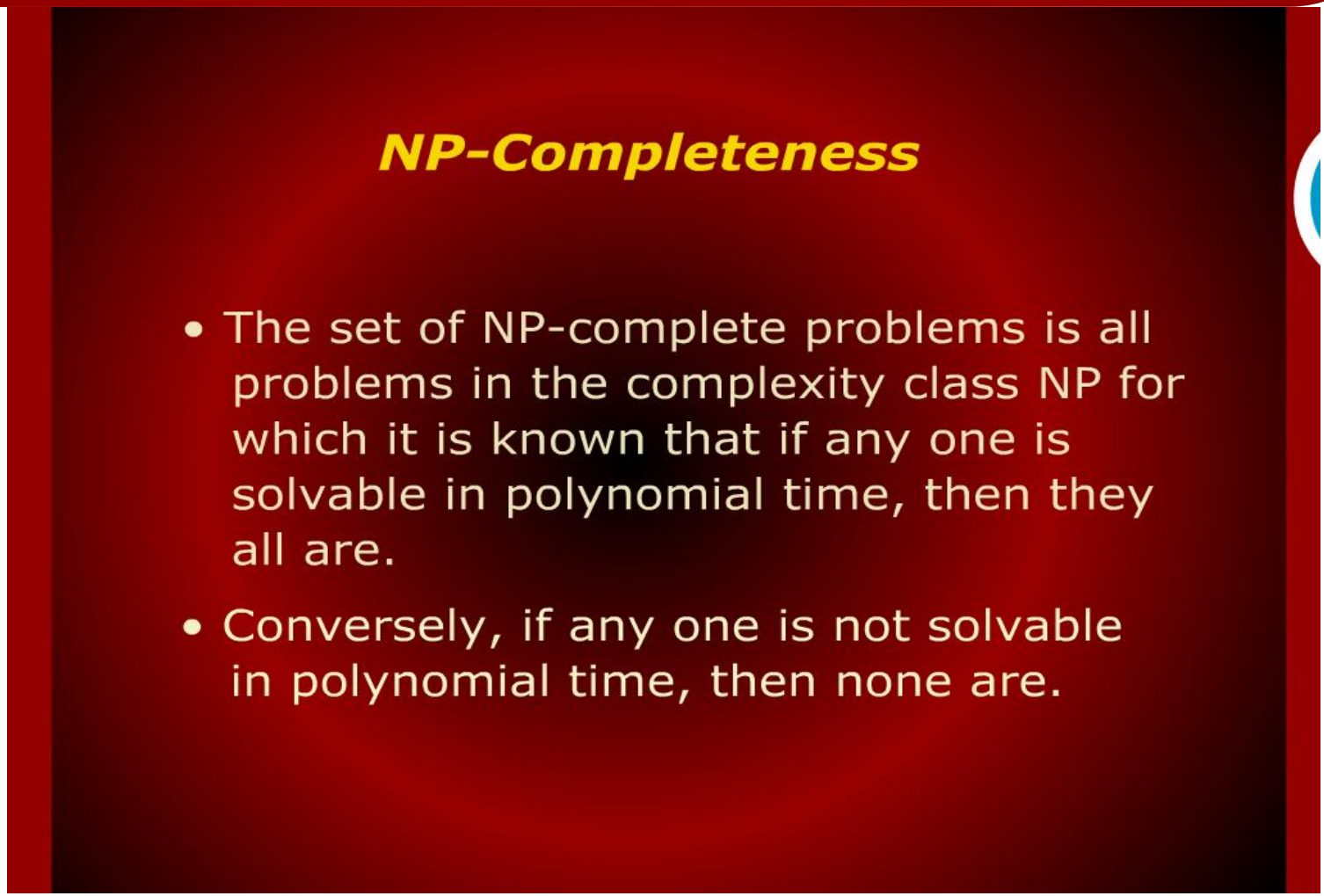
A decision problem L is NP-Hard if $L' \leq_P L$ for all $L' \in NP$.

Definition:

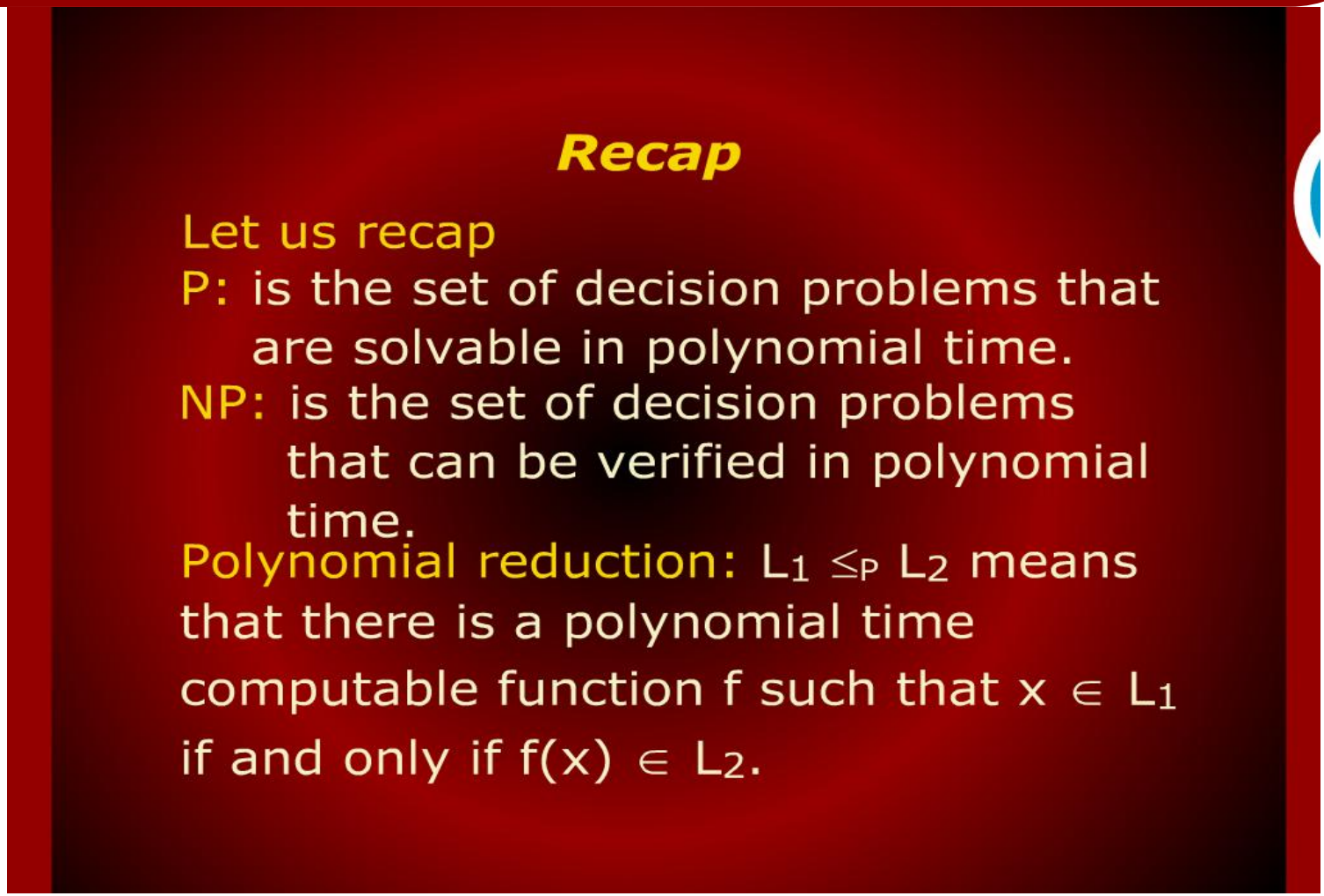
L is NP-complete if

1. $L \in NP$ and
2. $L' \leq_P L$ for some known NP-complete problem L'.

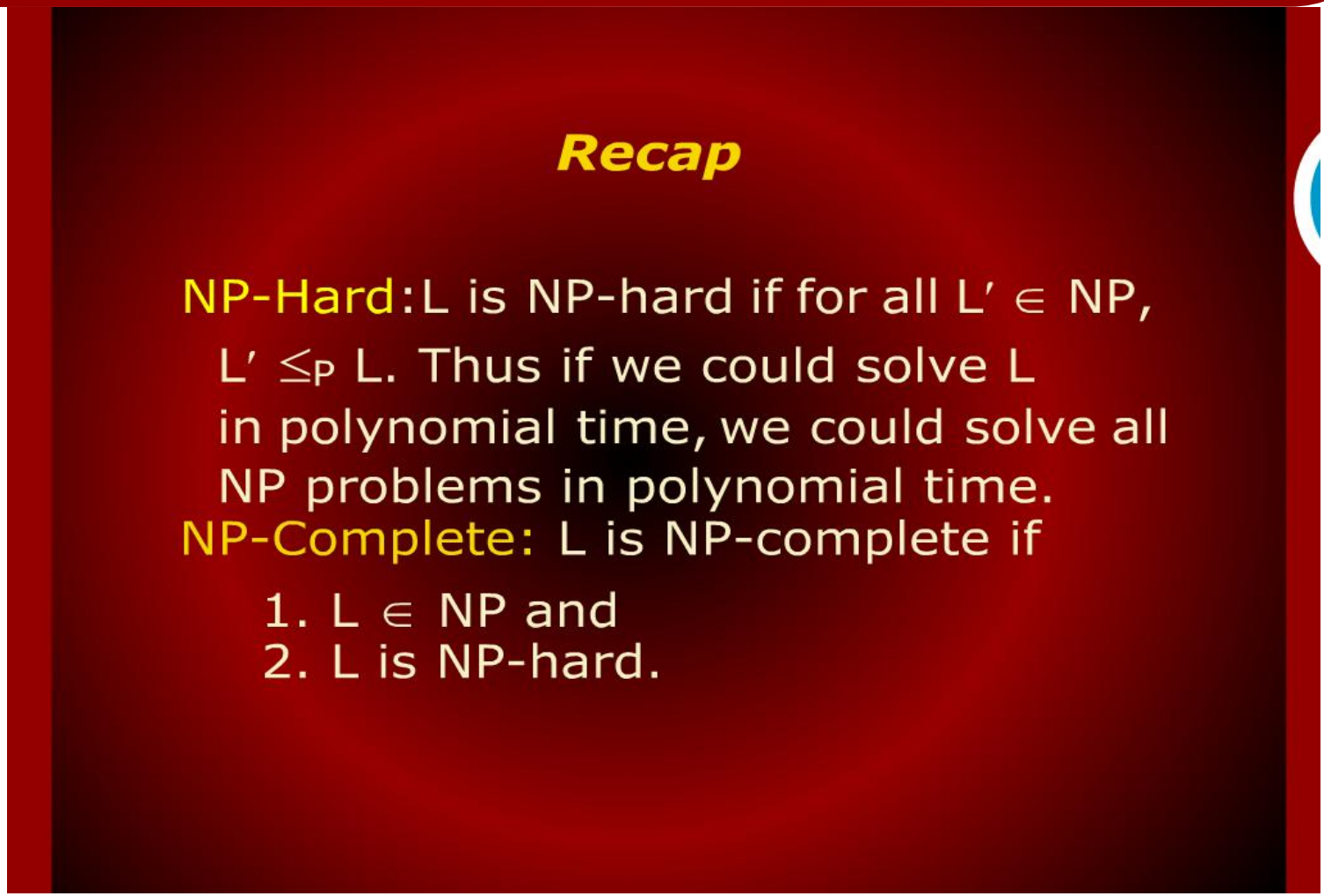
NP-Completeness



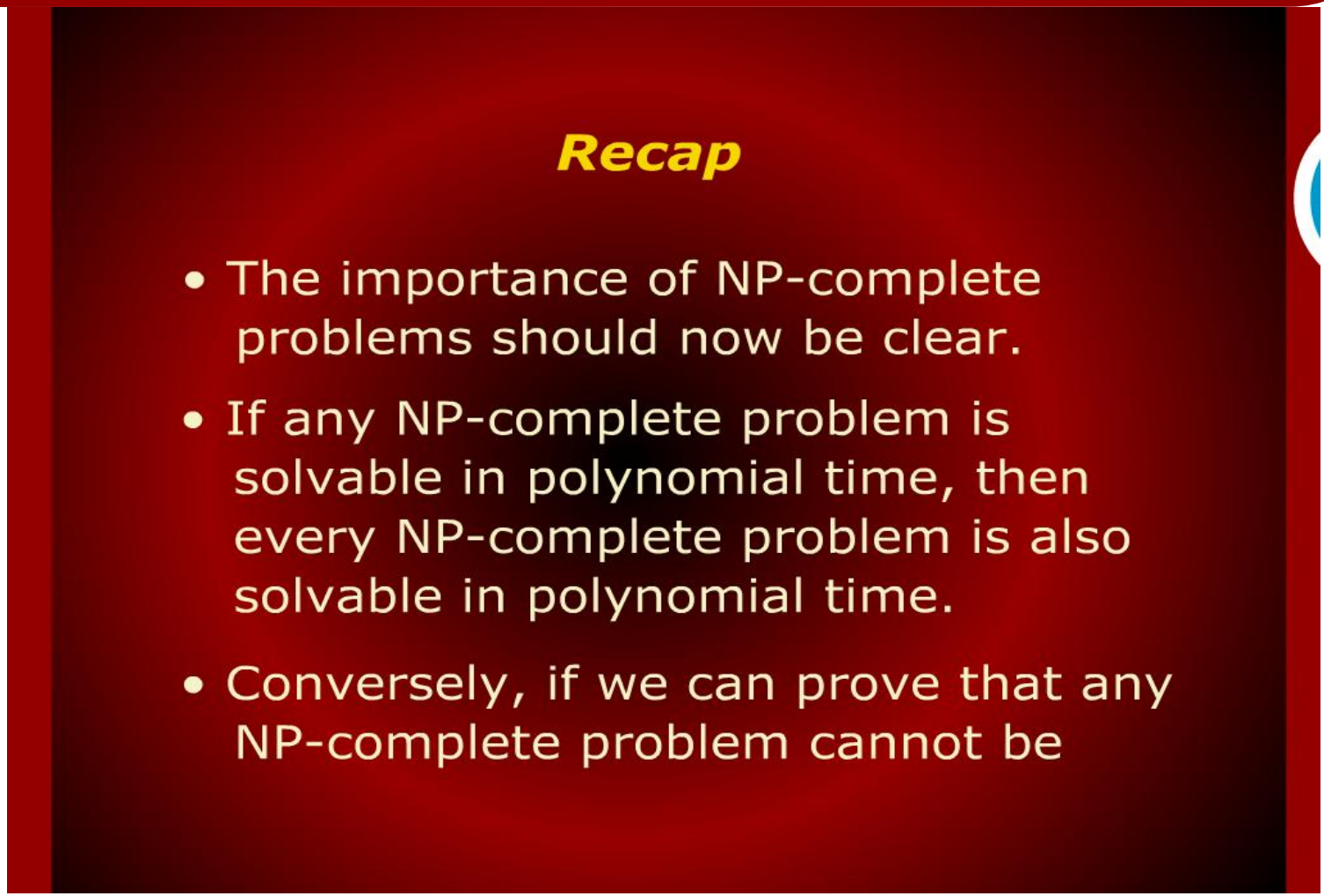
Recap


Recap
Let us recap
P: is the set of decision problems that are solvable in polynomial time.
NP: is the set of decision problems that can be verified in polynomial time.
Polynomial reduction: $L_1 \leq_P L_2$ means that there is a polynomial time computable function f such that $x \in L_1$ if and only if $f(x) \in L_2$.

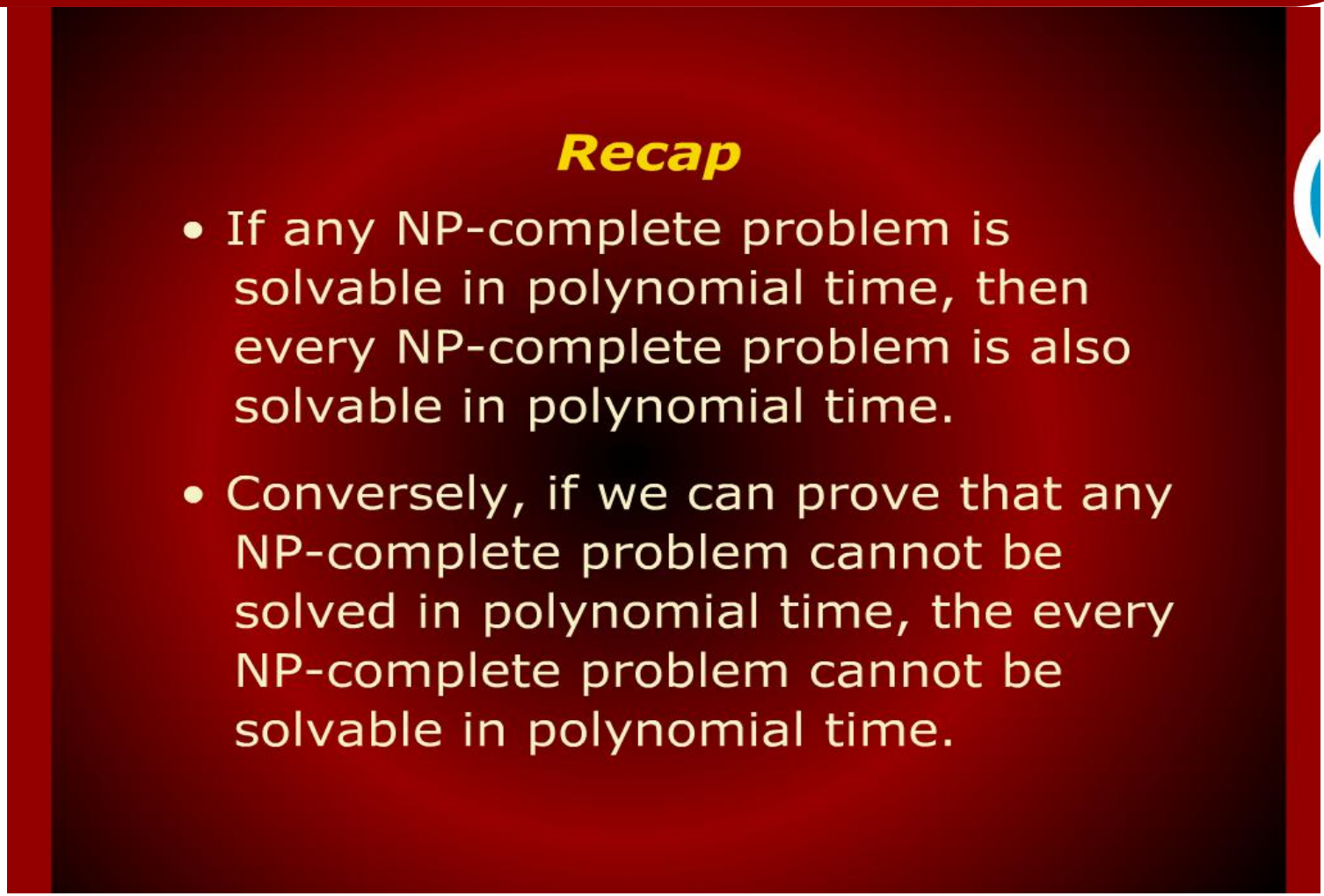
Recap


Recap
NP-Hard: L is NP-hard if for all L′ ∈ NP,
L′ ≤P L. Thus if we could solve L
in polynomial time, we could solve all
NP problems in polynomial time.
NP-Complete: L is NP-complete if
1. L ∈ NP and
2. L is NP-hard.

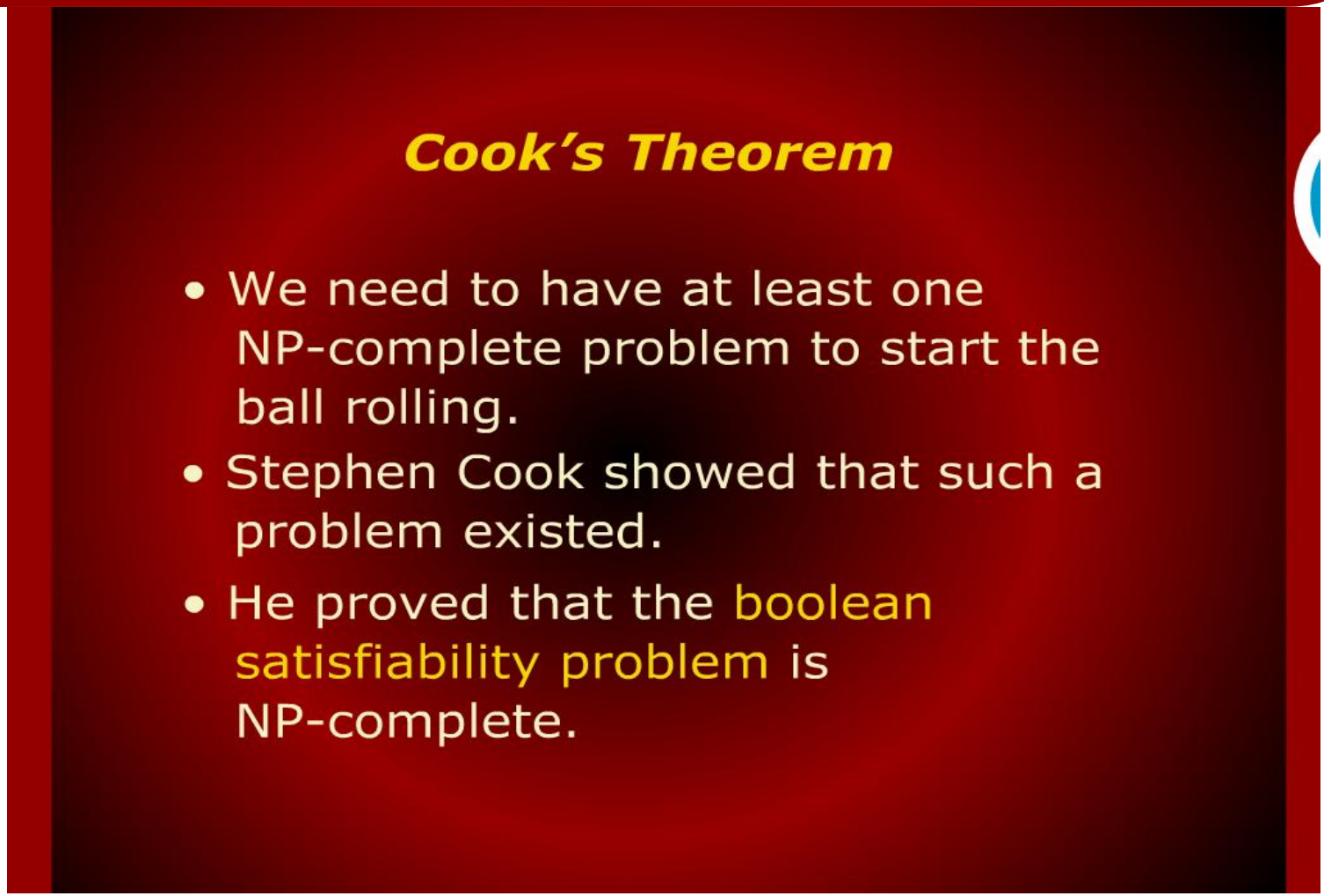
Recap


Recap
• The importance of NP-complete problems should now be clear.
• If any NP-complete problem is solvable in polynomial time, then every NP-complete problem is also solvable in polynomial time.
• Conversely, if we can prove that any NP-complete problem cannot be

Recap


Recap
• If any NP-complete problem is solvable in polynomial time, then every NP-complete problem is also solvable in polynomial time.
• Conversely, if we can prove that any NP-complete problem cannot be solved in polynomial time, the every NP-complete problem cannot be solvable in polynomial time.

cook's Theorem



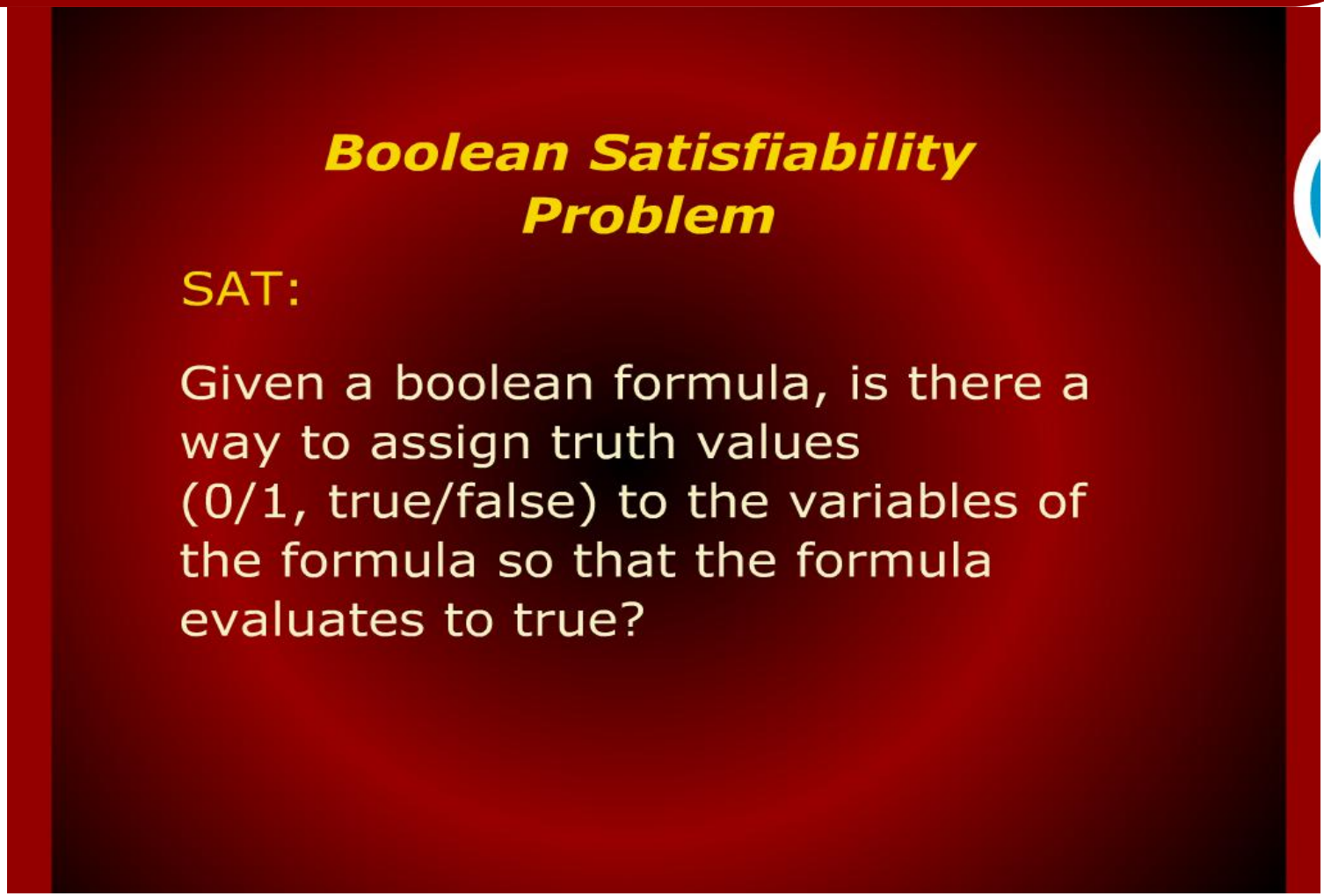
Boolean Satisfiability Problem

Boolean Satisfiability Problem

- A boolean formula is a logical formulation which consists of variables x_i.
- These variables appear in a logical expression using logical operations

1. negation of x: $\overline{x}$
2. boolean or: $(x \vee y)$
3. boolean and: $(x \wedge y)$

Boolean Satisfiability Problem

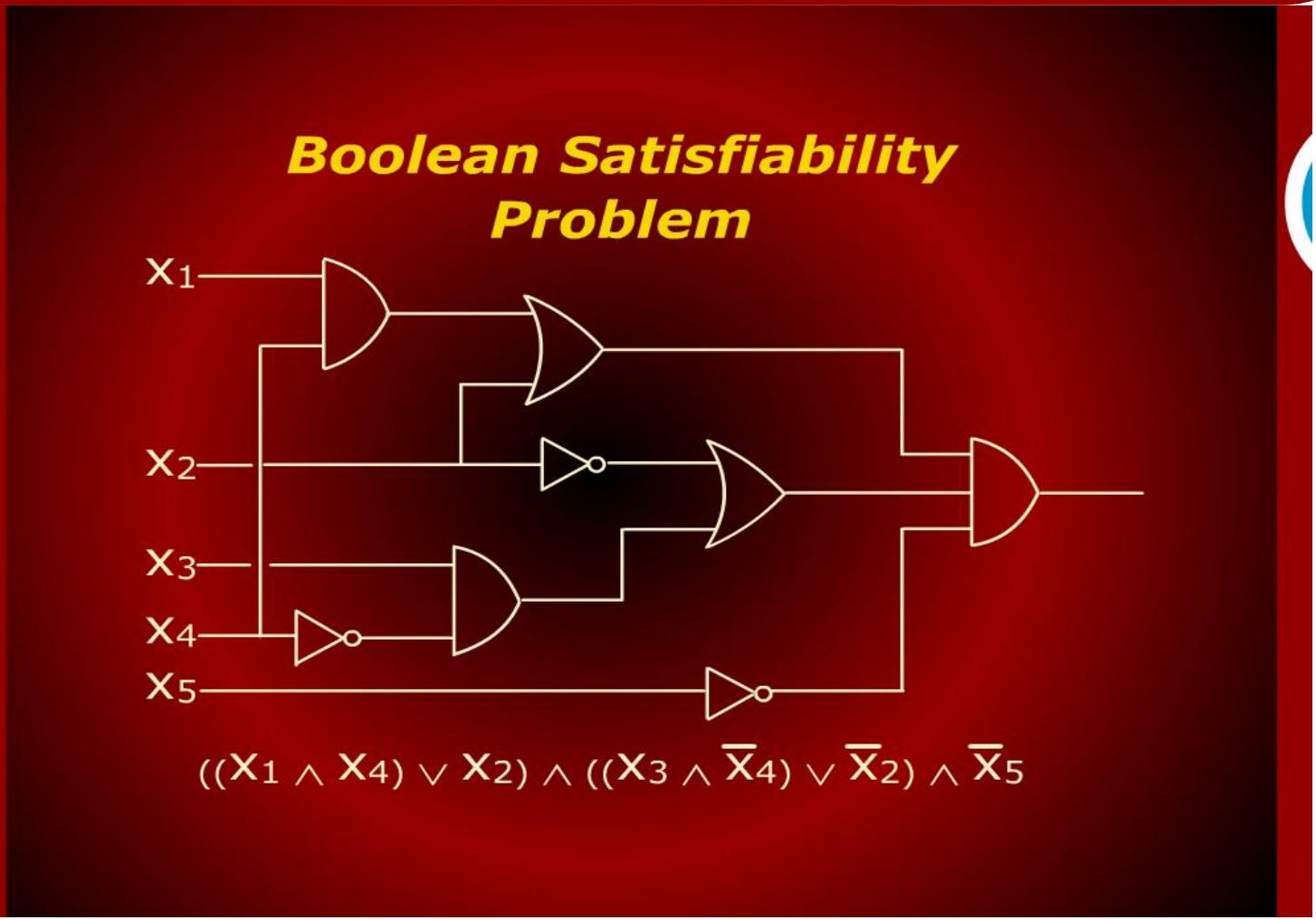


Boolean Satisfiability Problem

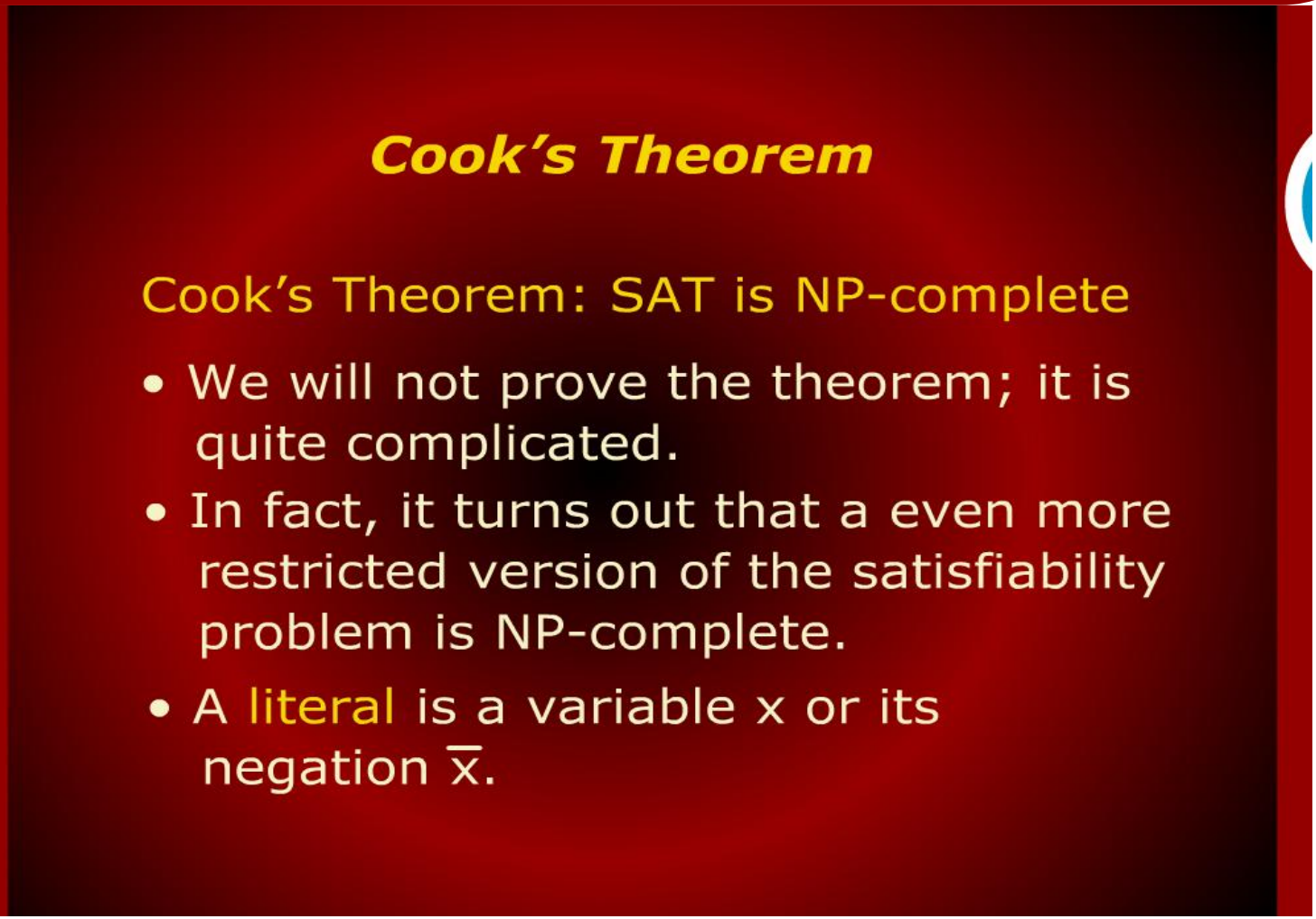
Boolean Satisfiability Problem

- For example
  $(x_1 \wedge (x_2 \vee \overline{x}_3)) \wedge ((\overline{x}_2 \wedge \overline{x}_3) \vee \overline{x}_1)$
  is satisfiable, by the assignment
  $x_1 = 1$, $x_2 = 0$ and $x_3 = 0$.
- On the other hand,
  $(\overline{x}_1 \vee (x_2 \wedge x_3)) \wedge (x_1 \vee (\overline{x}_2 \wedge \overline{x}_3)) \wedge (x_2 \vee x_3) \wedge (\overline{x}_2 \vee \overline{x}_3)$
  is not satisfiable.

Boolean Satisfiability Problem



cook's Theorem



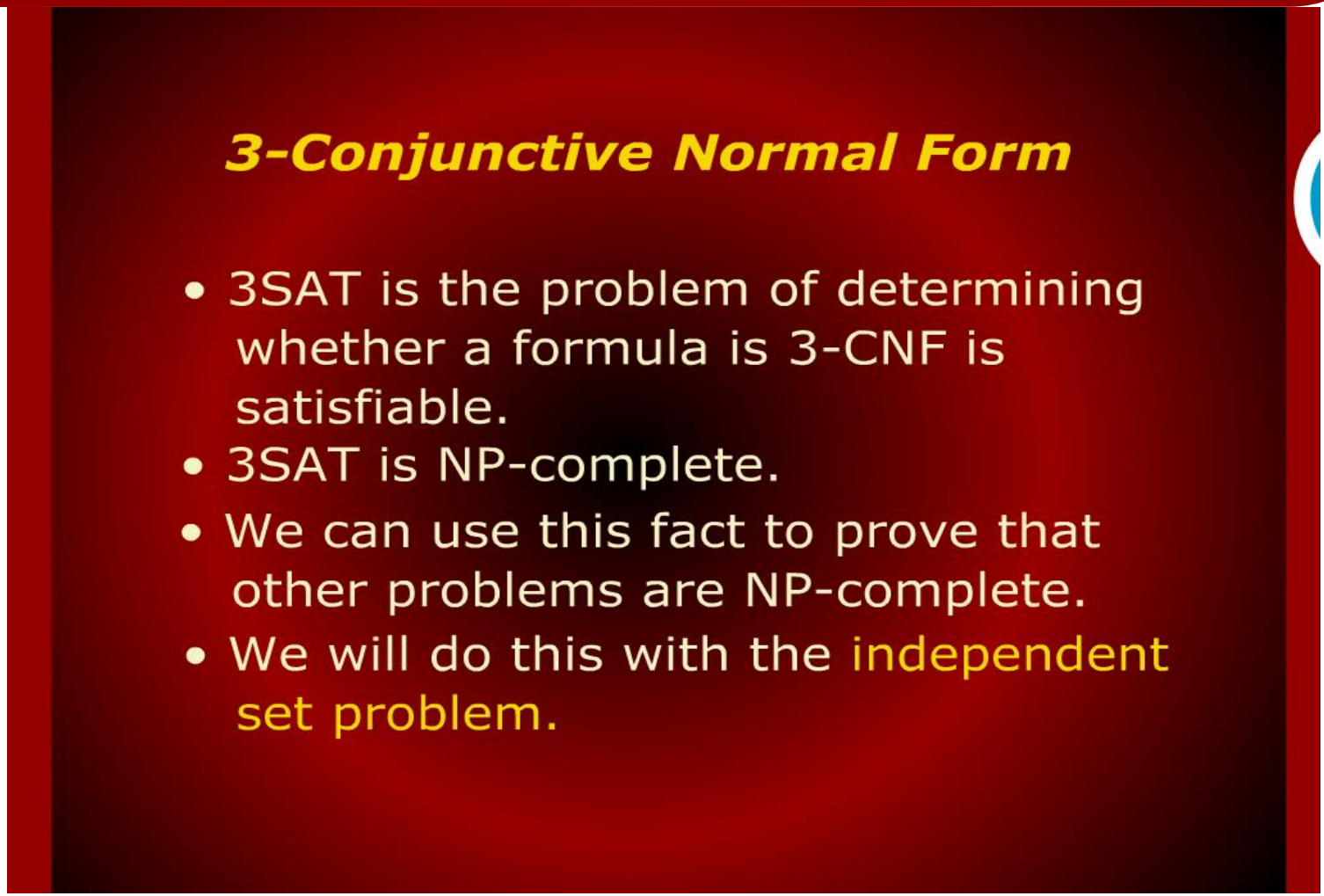
3-Conjuctive Normal Form

3-Conjunctive Normal Form

- A boolean formula is in 3-Conjunctive Normal Form (3-CNF) if it is the boolean-and of clauses where each clause is the boolean-or of exactly three literals.
- For example,

  $(x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_3 \vee x_4) \wedge (x_2 \vee \overline{x_3} \vee \overline{x_4})$ is in 3-CNF form

3-Conjuctive Normal Form



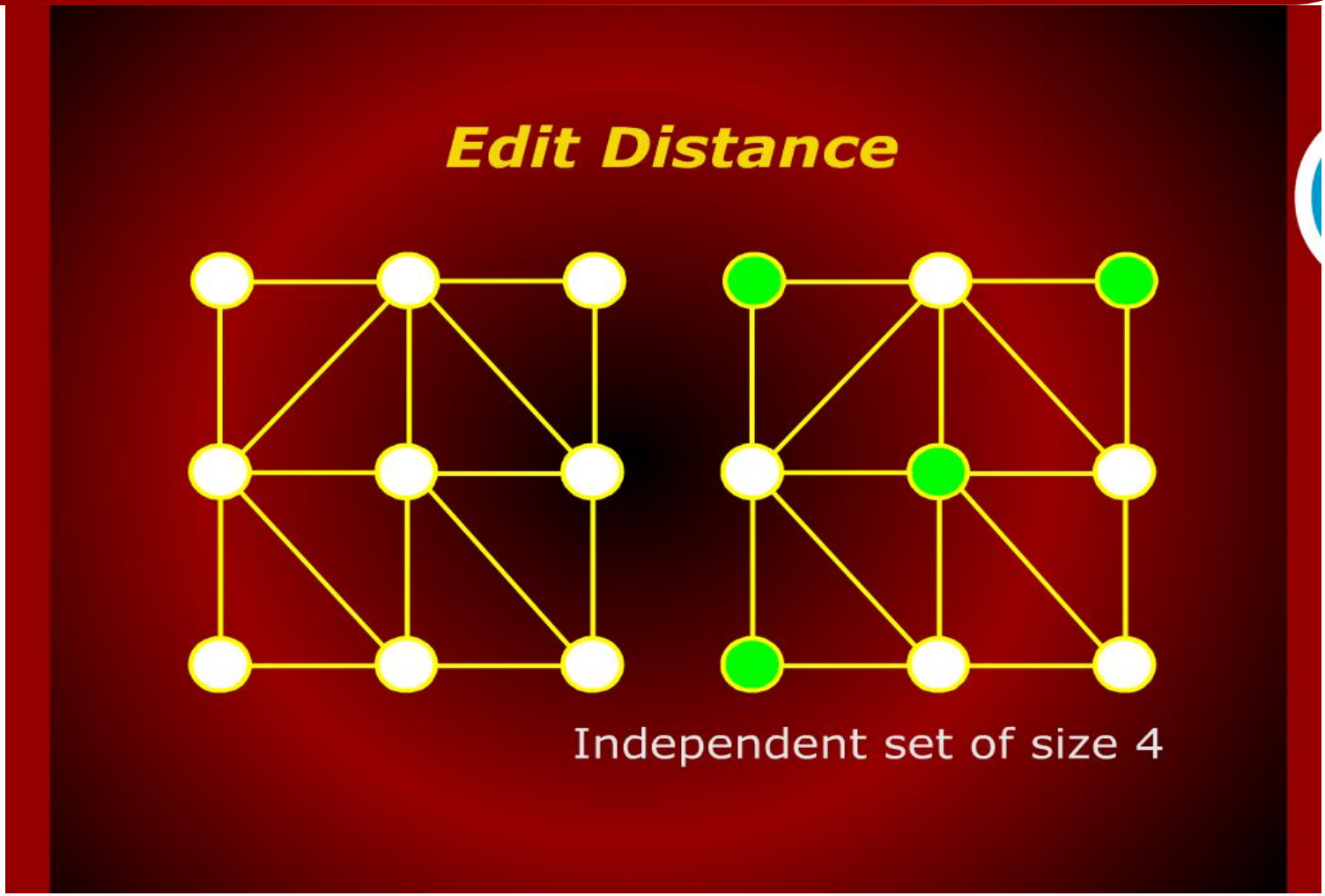
Independent Set Problem

Independent Set Problem

Independent Set Problem:

Given an undirected graph G = (V, E) and an integer k, does G contain a subset V′ of k vertices such that no two vertices in V′ are adjacent to each other.

Edit Distance

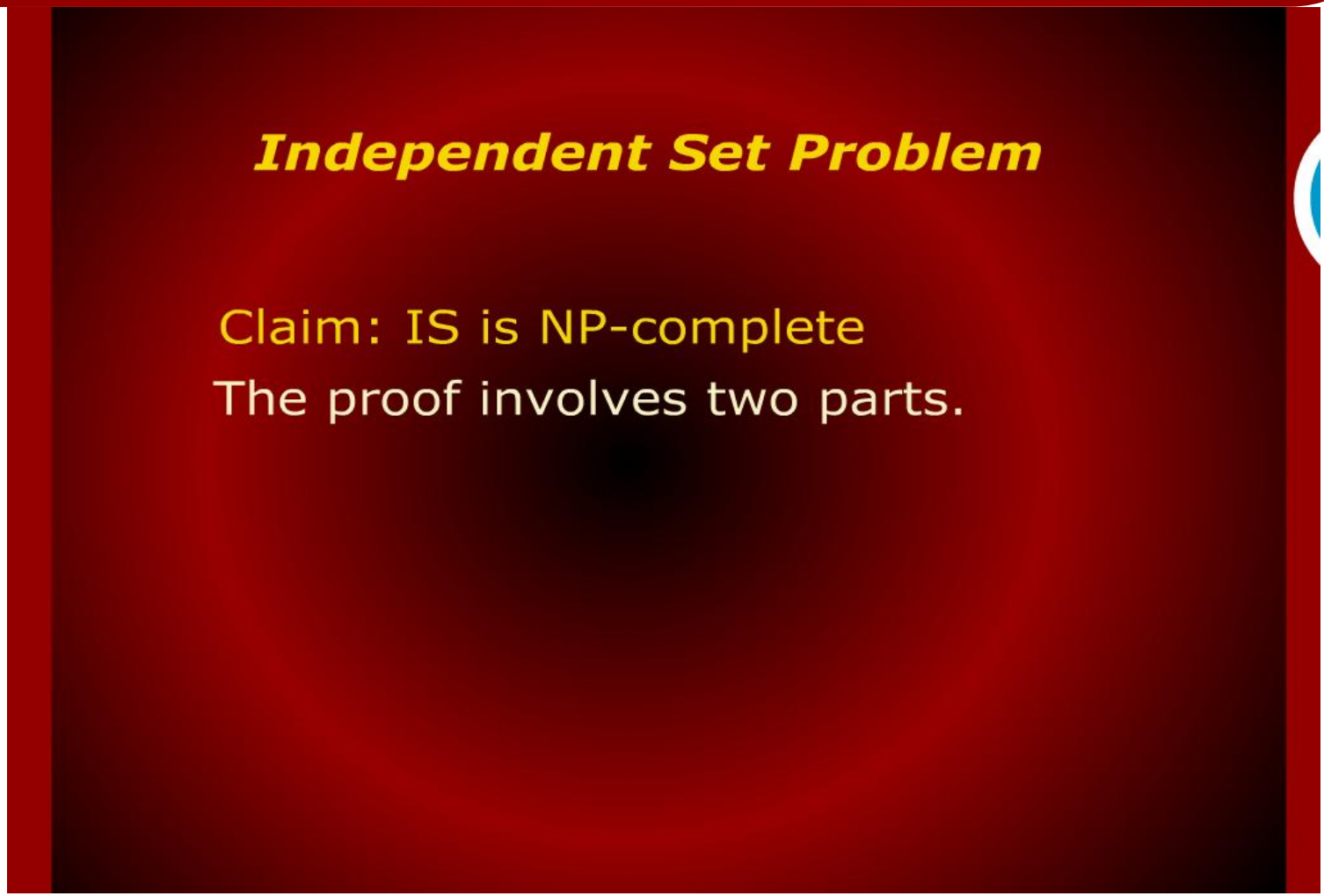


Independent Set Problem

Independent Set Problem

- The independent set problem arises when there is some sort of selection problem where there are mutual restrictions pairs that cannot both be selected.
- For example, a company dinner where an employee and hist/her immediate supervisor cannot both be invited.

Independent Set Problem



Independent Set Problem

Independent Set Problem

First: we need to show that IS $\in$ NP.

- The certificate consists of k vertices of V′.
- We simply verify that for each pair of vertices u, v $\in$ V′, there is no edge between them.
- Clearly, this can be done in polynomial time, by an inspection of the adjacency matrix.

Independent Set Problem

Independent Set Problem

Second: we need to establish that IS is NP-hard

- This can be done by showing that some known NP-compete (3SAT) is polynomial-time reducible to IS.
- That is, 3SAT $\leq_P$ IS.

Independent Set Problem

Independent Set Problem

Requirements:

3SAT: Each clause must contain at least one true valued literal.

IS: V′ must contain at least k vertices.

Independent Set Problem

Independent Set Problem

Restrictions:

3SAT: If x_i is assigned true, then $\overline{x_i}$ must be false and vice versa.

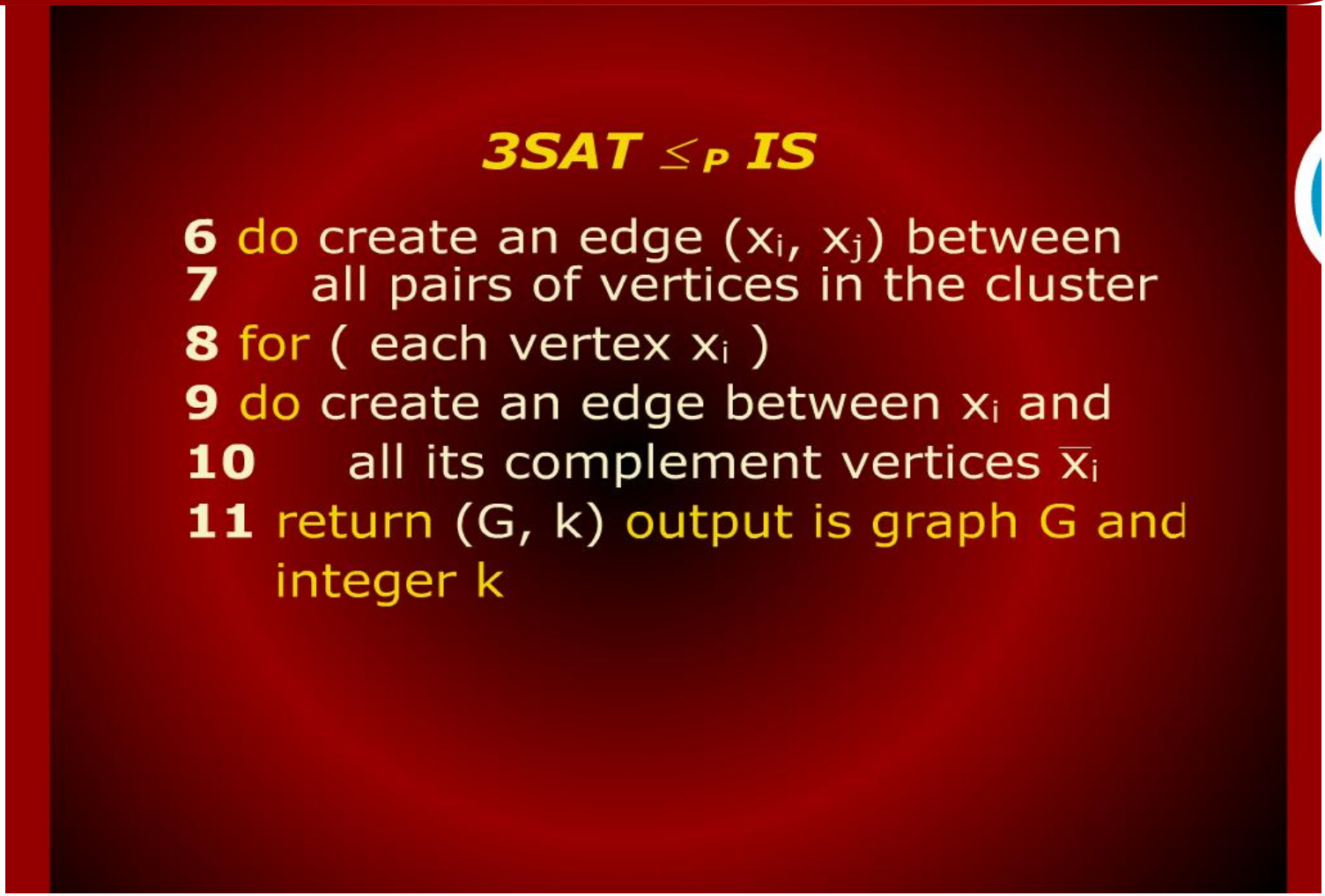
IS: If u is selected to be in V′ and v is a neighbor of u then v cannot be in V′.

3SAT

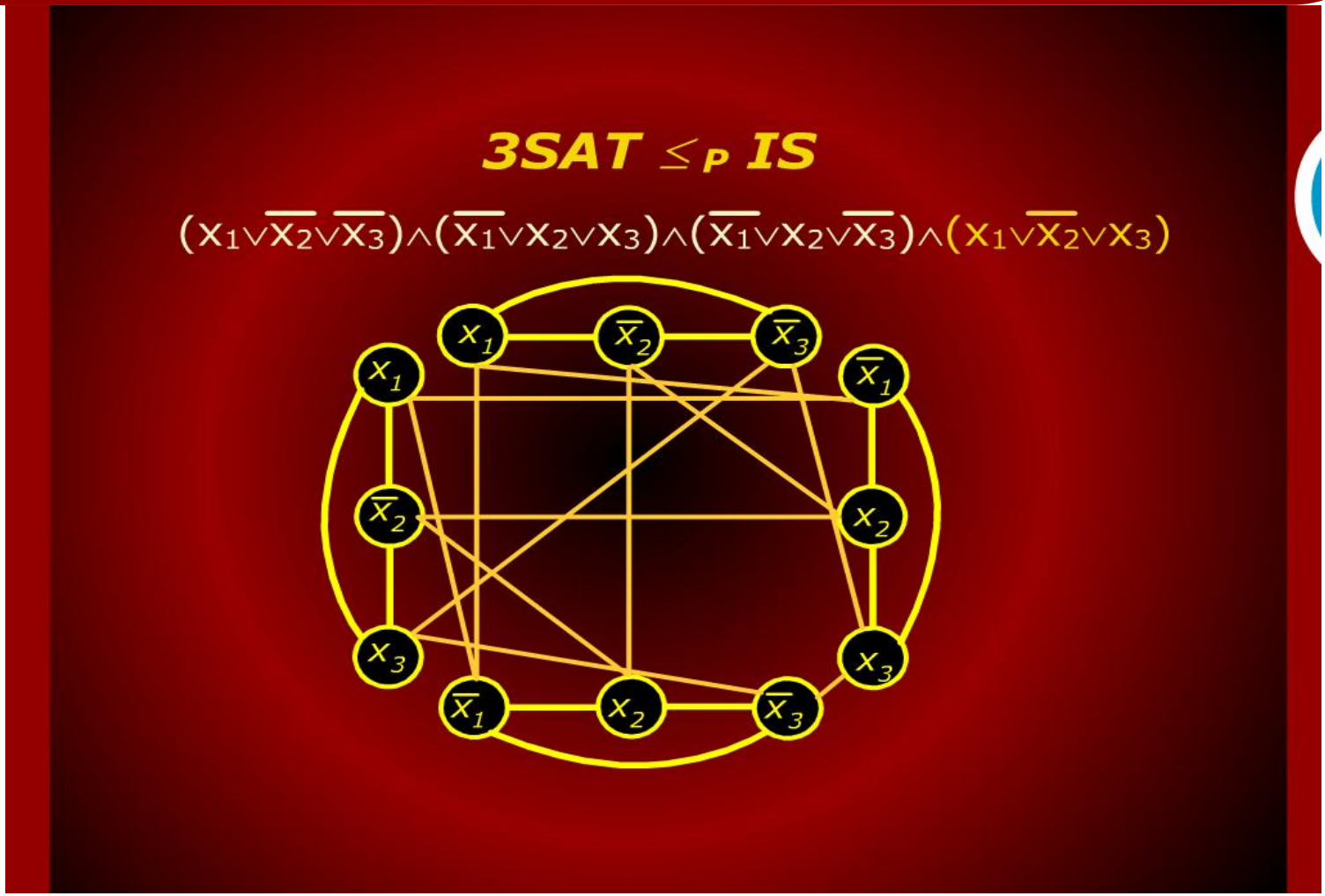
3SAT $\leq_P$ IS

```
3SAT-TO-IS(F:boolean formula in 3-CNF)
1 k ← number of clauses in F
2 for ( each clause C in F )
3 do create a clause cluster of
4       3 vertices from literals of C
5 for(each clause cluster (x1, x2,x3) )
6 do create an edge (xi, xj) between
7      all pairs of vertices in the cluster
8 for ( each vertex xi )
```

3SAT


3SAT ≤P IS
6 do create an edge (xi, xj) between
7 all pairs of vertices in the cluster
8 for (each vertex xi)
9 do create an edge between xi and
10 all its complement vertices x̄i
11 return (G, k) output is graph G and
integer k

3SAT


3SAT ≤P IS
(x1∨x̄2∨x̄3)∧(x̄1∨x2∨x3)∧(x̄1∨x2∨x̄3)∧(x1∨x̄2∨x3)
x1
x̄2
x̄3
x1
x̄1
x̄2
x2
x3
x3
x̄1
x2
x̄3

3SAT

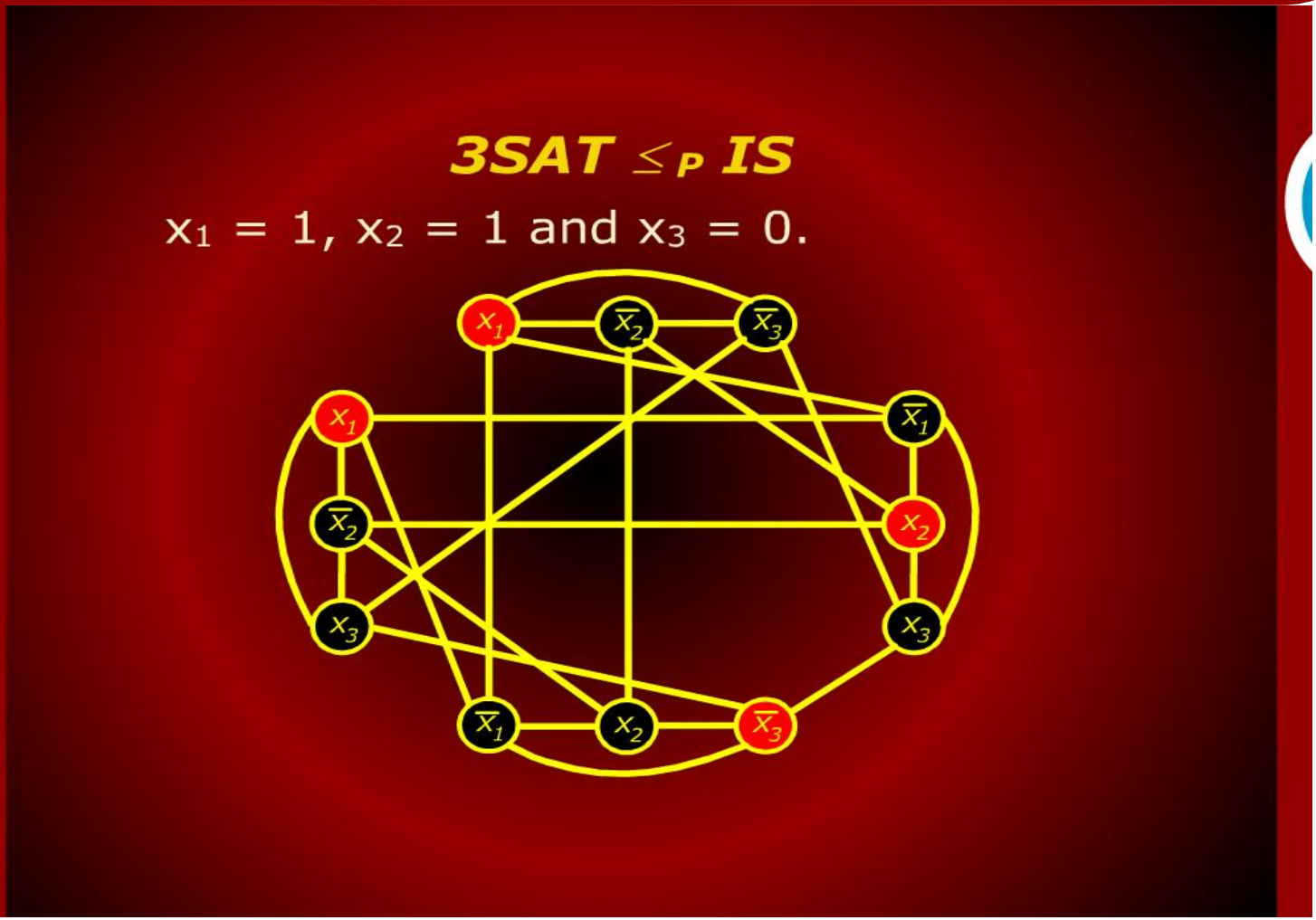
3SAT $\leq_P$ IS

$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3)$

The boolean formula is satisfied by the assignment $x_1 = 1$, $x_2 = 1$ $x_3 = 0$

By selecting vertices corresponding to true literals in each clause, we get an independent set of size $k = 4$.

3SAT



3SAT

$3SAT \leq_P IS$

- The transformation clearly runs in polynomial time.
- Also observe that the the reduction had no knowledge of the solution to either problem.
- Computing the solution to either will require exponential time.
- Instead, the reduction simple translated the input from one problem into an equivalent input to the other problem.

Coping with NP-Completeness

Coping with NP-Completeness

Use brute-force search: Even on the fastest parallel computers this approach is viable only for the smallest instance of these problems.

Heuristics: A heuristic is a strategy for producing a valid solution but there are no guarantees how close it to optimal. This is worthwhile if all else fails.

Graph Coloring

Boolean Satisflability problem